

Payment Service

Documentação de Arquitectura

Sistema de pagamentos resiliente para Moçambique
Spring Boot 3 · PostgreSQL · RabbitMQ · Kubernetes

Stack	Versão	Função
Spring Boot	3.x	Framework principal
PostgreSQL	15+	Base de dados relacional
RabbitMQ	3.12+	Mensageria assíncrona
Kubernetes	1.28+	Orquestração de containers
Java	17+	Linguagem de programação

1. Visão Geral

O Payment Service é um sistema de pagamentos resiliente, desenhado especificamente para o contexto de Moçambique: ambientes com baixa conectividade, integrações com provedores externos como MPesa e e-Mola, e necessidade de alta disponibilidade mesmo em condições de rede instável.

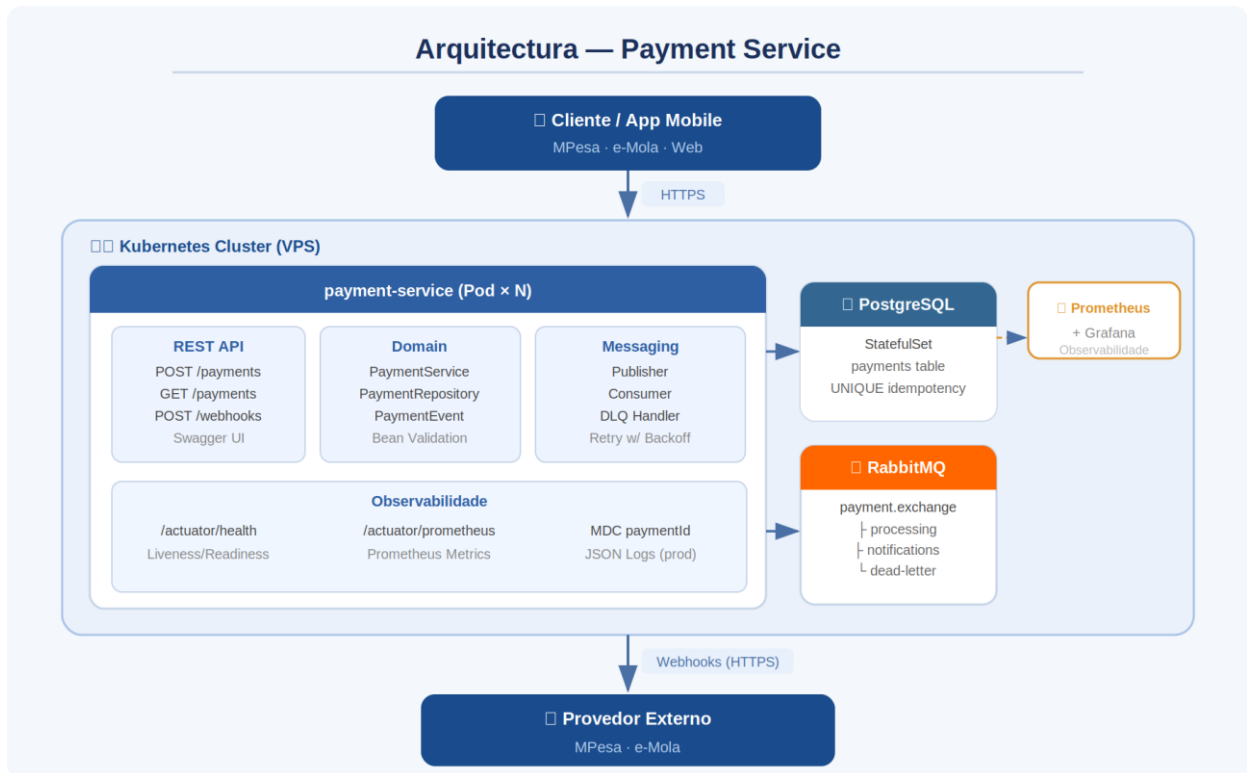
O sistema adopta uma arquitectura de microsserviços assíncrona, baseada em eventos, garantindo que nenhum pagamento seja perdido ou duplicado, mesmo em cenários de falha de rede, reinício de serviços ou sobrecarga de provedores.

Princípios de Design

- Resiliência por omissão — todos os componentes assumem que a rede pode falhar
- Idempotência garantida — o mesmo pagamento nunca é processado duas vezes
- Observabilidade integrada — métricas, logs e health checks prontos para produção
- Segurança first — containers não-root, segredos via Kubernetes Secrets

2. Diagrama de Arquitectura

O diagrama abaixo ilustra os componentes principais e as suas interações dentro do cluster Kubernetes:



□ Todos os componentes correm dentro de um cluster Kubernetes em VPS. O único ponto de entrada externo é o endpoint HTTPS para clientes e o webhook do provedor.

3. Componentes

3.1 API Layer

Componente	Responsabilidade
PaymentController	POST /api/v1/payments · GET /api/v1/payments/{id}
WebhookController	POST /webhooks/provider — callback assíncrono do provedor
GlobalExceptionHandler	Mapeia excepções para ProblemDetail (RFC 9457)
DTOs	PaymentRequest · WebhookRequest · PaymentResponse

3.2 Domain Layer

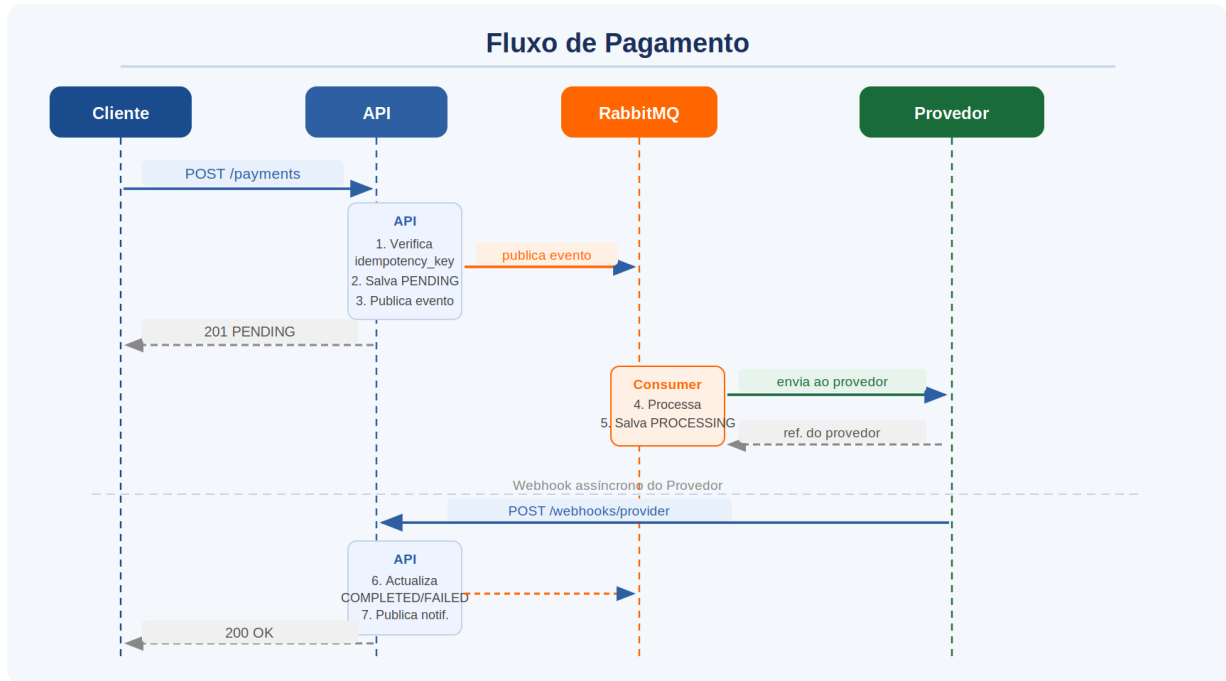
Componente	Responsabilidade
PaymentService	Orquestra criação, processamento e webhook
Payment	Entidade JPA com auditoria automática (@CreatedDate, @LastModifiedDate)
PaymentStatus	PENDING → PROCESSING → COMPLETED FAILED CANCELLED
PaymentRepository	Queries por id, idempotencyKey, providerRef, status+retryCount

3.3 Messaging Layer

Componente	Responsabilidade
RabbitMQConfig	Declaração de exchanges, queues e bindings
PaymentEventPublisher	Publica eventos: processing · completed · failed
PaymentEventConsumer	Consume: processing · notifications · dead-letter

4. Fluxo de Pagamento

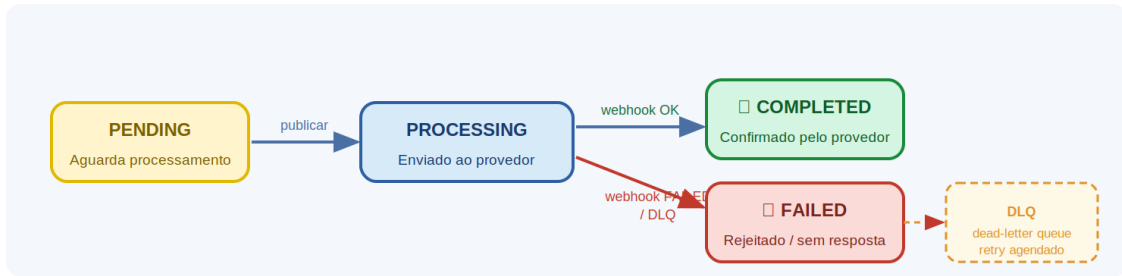
O diagrama de sequência abaixo detalha o ciclo completo de um pagamento, desde a criação pelo cliente até à confirmação final pelo provedor via webhook:



Passos do Fluxo

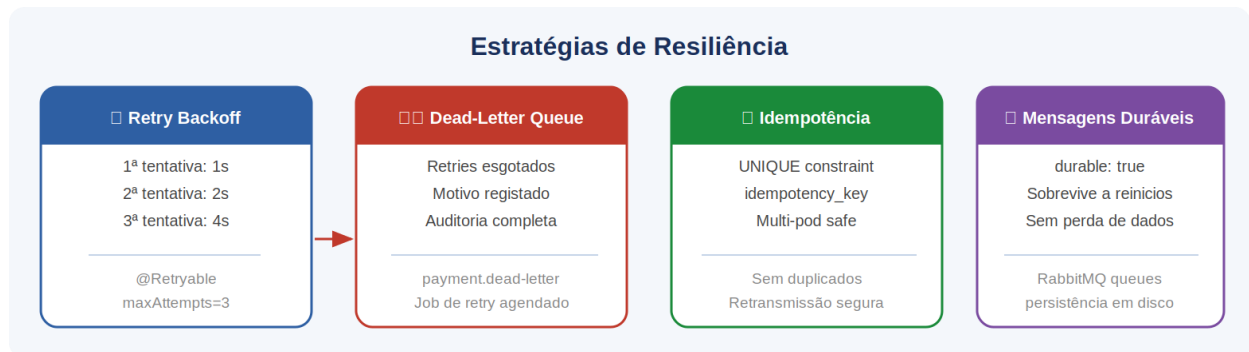
#	Actor	Acção
1	Cliente → API	POST /payments com idempotencyKey único
2	API	Verifica idempotency_key — previne duplicados
3	API	Salva pagamento com status PENDING
4	API → RabbitMQ	Publica evento de processamento
5	Consumer → Provedor	Envia pedido ao MPesa/e-Mola, actualiza para PROCESSING
6	Provedor → API	Webhook com COMPLETED ou FAILED
7	API	Actualiza status final, publica notificação

5. Ciclo de Vida do Pagamento



Status	Descrição	Próximo Estado
PENDING	Criado, aguarda processamento pelo provedor	PROCESSING
PROCESSING	Enviado ao provedor, aguarda confirmação	COMPLETED · FAILED
COMPLETED	Confirmado com sucesso pelo provedor via webhook	Terminal
FAILED	Rejeitado pelo provedor ou sem resposta após retries	DLQ → retry
CANCELLED	Cancelado manualmente ou por timeout de negócio	Terminal

6. Estratégias de Resiliência



6.1 Retry com Backoff Exponencial

Cada chamada ao provedor é protegida por retry automático com backoff exponencial:

```
@Retryable(maxAttempts = 3, backoff = @Backoff(delay = 1000, multiplier = 2.0))
// Tentativas: 1s → 2s → 4s → DLQ
```

Após esgotar os retries, o pagamento é enviado para a Dead-Letter Queue com o motivo de falha registado. Um job agendado pode reprocessar pagamentos em DLQ.

6.2 Idempotência

A constraint UNIQUE na coluna idempotency_key a nível de base de dados previne pagamentos duplicados, mesmo com múltiplos pods a correr em paralelo ou retransmissões de rede.

6.3 Mensagens Duráveis

Todas as queues RabbitMQ são configuradas com durable: true, garantindo que as mensagens sobrevivem a reinícios do broker sem perda de dados.

7. Observabilidade

Componente	Endpoint / Mecanismo
Health check	GET /actuator/health (liveness + readiness)
Métricas Prometheus	GET /actuator/prometheus
Counter payments.created	Incrementado por cada pagamento processado com sucesso
Counter payments.provider.failed	Incrementado por cada falha permanente do provedor
MDC paymentId	Presente em todos os logs dentro do contexto de um pagamento
Logs JSON (prod)	Via logstash-logback-encoder com profile prod

SLIs / SLOs Sugeridos

Métrica	Objectivo	Notas
Disponibilidade	≥ 99.5%	Uptime do endpoint POST /payments
Latência p99	< 500ms	Excluindo tempo do provedor externo
Taxa de erro 5xx	< 1%	Respostas com erro de servidor

8. Segurança

Medida	Implementação
Container não-root	Executa como appuser — reduz superfície de ataque
Stack traces	Nunca expostos na resposta HTTP (include-stacktrace: never)
Credenciais	Injectadas via variáveis de ambiente / Kubernetes Secrets
Validação de input	Bean Validation: @NotBlank · @DecimalMin · @Size

9. Decisões de Arquitectura (ADRs)

As seguintes Architecture Decision Records documentam as principais escolhas técnicas do sistema:

ADR	Decisão	Ficheiro
ADR-001	PostgreSQL como base de dados principal	docs/adr/ADR-001-postgresql.pdf
ADR-002	RabbitMQ para processamento assíncrono	docs/adr/ADR-002-rabbitmq.pdf
ADR-003	Idempotência via Idempotency-Key	docs/adr/ADR-003-idempotencia.pdf

Consulte a pasta docs/adr/ para os ADRs detalhados com contexto, opções consideradas e consequências de cada decisão.